

Python Dictionary Revision Chart

This revision sheet covers Python dictionary properties, mutability, built-in functions, methods, operations, and the note about indexing and slicing.[cite:71][cite:72][cite:79]

Dictionary chart

Section	Number	Topic	Explanation	Example	Output
Properties	1	Syntax	Dictionary is written with curly braces { } using key: value pairs. [cite:71] [cite:79]	<code>d = {"a": 1, "b": 2}</code>	<code>{'a': 1, 'b': 2}</code>
Properties	2	Empty dictionary	Empty dictionary can be written as { } or <code>dict()</code> . [cite:76] [cite:79]	<code>d = {}</code>	<code>{}</code>
Properties	3	Mutability	Dictionary is mutable, so items can be added, updated, or removed after creation. [cite:76] [cite:79]	<code>d = {"a": 1}</code> then <code>d["b"] = 2</code>	<code>{'a': 1, 'b': 2}</code>
Properties	4	Key-value pairs	Each item in a dictionary is stored as a key and its value. [cite:71] [cite:73]	<code>{"name": "Ram", "age": 20}</code>	Key-value data

Section	Number	Topic	Explanation	Example	Output
Properties	5	Unique keys	Duplicate keys are not allowed; the later value replaces the earlier one. [cite:72] [cite:73]	<code>d = {"a": 1, "a": 5}</code>	<code>{'a': 5}</code>
Properties	6	Keys must be immutable	Keys should be immutable types like string, number, or tuple. [cite:73] [cite:77] [cite:80]	<code>d = {(1,2): "point"}</code>	Valid
Properties	7	Ordered	Dictionaries keep insertion order in modern Python. [cite:75] [cite:76]	<code>{"x":1, "y":2, "z":3}</code>	Order preserved
Properties	8	Access by key	Values are accessed using key names, not numeric index positions. [cite:71] [cite:79]	<code>d = {"a": 10} print(d["a"])</code>	10
Properties	9	No normal slicing	Dictionary does not support slicing with start, stop, and step like sequences do.[cite:79] [cite:80]	<code>d = {"a":1,"b":2} d[0:1]</code>	Error
Properties	10	Membership testing	<code>in</code> checks keys in a dictionary, not values. [cite:79]	<code>print("a" in {"a":1,"b":2})</code>	True

Section	Number	Topic	Explanation	Example	Output
Built-in Function	1	<code>len()</code>	Returns total number of key-value pairs. [cite:79]	<code>print(len({"a":1,"b":2}))</code>	2
Built-in Function	2	<code>type()</code>	Returns the data type of dictionary. [cite:72]	<code>print(type({"a":1}))</code>	<class 'dict'>
Built-in Function	3	<code>dict()</code>	Creates a new dictionary. [cite:71] [cite:76]	<code>d = dict(a=1, b=2)</code>	<code>{'a': 1, 'b': 2}</code>
Built-in Function	4	<code>sorted()</code>	Returns a sorted list of keys by default. [cite:79]	<code>print(sorted({"b":2,"a":1}))</code>	<code>['a', 'b']</code>
Built-in Function	5	<code>in</code>	Checks whether a key exists. [cite:79]	<code>print("a" in {"a":1})</code>	True
Built-in Function	6	<code>not in</code>	Checks whether a key does not exist. [cite:79]	<code>print("z" not in {"a":1})</code>	True
Method	1	<code>get()</code>	Returns the value for a key without an error if the key is missing. [cite:71] [cite:79]	<code>print({"a":1}.get("a"))</code>	1
Method	2	<code>keys()</code>	Returns all keys of the dictionary. [cite:71] [cite:76]	<code>print({"a":1,"b":2}.keys())</code>	<code>dict_keys(['a', 'b'])</code>
Method	3	<code>values()</code>	Returns all values of the dictionary. [cite:74] [cite:76]	<code>print({"a":1,"b":2}.values())</code>	<code>dict_values([1, 2])</code>

Section	Number	Topic	Explanation	Example	Output
Method	4	<code>items()</code>	Returns all key-value pairs. [cite:71] [cite:76] [cite:78]	<code>print({"a":1,"b":2}.items())</code>	<code>dict_items([('a', 1), ('b', 2)])</code>
Method	5	<code>update()</code>	Adds items from another dictionary or updates existing keys. [cite:71] [cite:76]	<code>d={"a":1} d.update({"b":2}) print(d)</code>	<code>{'a': 1, 'b': 2}</code>
Method	6	<code>pop()</code>	Removes the item with the given key and returns its value. [cite:76]	<code>d={"a":1,"b":2} print(d.pop("a"))</code>	<code>1</code>
Method	7	<code>popitem()</code>	Removes and returns the last inserted key-value pair. [cite:75] [cite:76]	<code>d={"a":1,"b":2} print(d.popitem())</code>	<code>('b', 2)</code>
Method	8	<code>clear()</code>	Removes all items from the dictionary. [cite:76]	<code>d={"a":1} d.clear() print(d)</code>	<code>{}</code>
Method	9	<code>copy()</code>	Creates a copy of the dictionary. [cite:76]	<code>d={"a":1} e=d.copy() print(e)</code>	<code>{'a': 1}</code>
Method	10	<code>setdefault()</code>	Returns the value of a key; inserts the key with a default value if missing. [cite:71]	<code>d={"a":1} print(d.setdefault("b", 0))</code>	<code>0</code>

Section	Number	Topic	Explanation	Example	Output
Operation	1	Add/Update using <code>=</code>	A new key adds an item; an existing key updates the value. [cite:71] [cite:76]	<code>d={"a":1} d["b"]=2</code>	<code>{'a': 1, 'b': 2}</code>
Operation	2	Delete using <code>del</code>	Deletes a key-value pair. [cite:76]	<code>d={"a":1,"b":2} del d["a"]</code>	<code>{'b': 2}</code>
Operation	3	Membership <code>in</code>	Checks whether a key is present. [cite:79]	<code>print("b" in {"a":1,"b":2})</code>	<code>True</code>
Operation	4	Membership not <code>in</code>	Checks whether a key is absent. [cite:79]	<code>print("x" not in {"a":1})</code>	<code>True</code>

Indexing and slicing note

Dictionary values are accessed by key, not by numeric index, so normal sequence-style indexing is not the main access pattern.[cite:71][cite:79] Dictionaries also do not support slicing with `start:stop:step` because slicing belongs to sequence types such as strings, lists, and tuples.[cite:79][cite:80]

Mutability note

Dictionary is mutable because data can be added, changed, and deleted after creation using assignment, `update()`, `pop()`, `popitem()`, `clear()`, and `del`. [cite:71][cite:76][cite:79]